

1. Write a menu-driven program in C to perform various operations on a Single Linked List (Create, Display, Display Alternate, Find Max/Min, Insert, Delete, Reverse, Find Middle).

**Source Code:**

```
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *next;
};

struct node* create_list();
void display(struct node *head);
void display_alternate(struct node *head);
void maxmin(struct node *head);
struct node* insert_node(struct node *head);
struct node* delete_node(struct node *head);
struct node* reverse_list(struct node *head);
void find_middle(struct node *head);
void free_list(struct node *head);

int main() {
    struct node *head = NULL;
    int choice;

    while (1) {
        printf("\n1. Create Linked List\n");
        printf("2. Display List\n");
        printf("3. Display Alternate Nodes\n");
        printf("4. Find Largest and Smallest Elements\n");
        printf("5. Insert a Node\n");
        printf("6. Delete a Node\n");
        printf("7. Reverse Linked List\n");
        printf("8. Find Middle Element\n");
        printf("9. Exit\n");
        printf("Enter your choice (1-9): ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                if (head != NULL) {
                    free_list(head);
                    head = NULL;
                }
                head = create_list();
                break;
            case 2:
                display(head);
                break;
            case 3:
                display_alternate(head);
                break;
```

```

        case 4:
            maxmin(head);
            break;
        case 5:
            head = insert_node(head);
            break;
        case 6:
            head = delete_node(head);
            break;
        case 7:
            head = reverse_list(head);
            display(head);
            break;
        case 8:
            find_middle(head);
            break;
        case 9:
            free_list(head);
            exit(0);
        default:
            printf("Invalid choice! Please select 1 to 9.\n");
    }
}
return 0;
}

```

```

struct node* create_list() {
    struct node *head = NULL, *ptr = NULL, *newNode = NULL;
    char ch;

    do {
        newNode = (struct node*) malloc(sizeof(struct node));
        if (newNode == NULL) {
            return head;
        }

        printf("Enter Data: ");
        scanf("%d", &newNode->data);
        newNode->next = NULL;

        if (head == NULL) {
            head = newNode;
            ptr = newNode;
        } else {
            ptr->next = newNode;
            ptr = ptr->next;
        }

        printf("Do you have another data? (y/n): ");
        scanf(" %c", &ch);
    } while (ch == 'y' || ch == 'Y');

    return head;
}

```

```

void display(struct node *head) {
    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }
    struct node *ptr = head;
    printf("List elements: ");
    while (ptr != NULL) {
        printf("%d ", ptr->data);
        ptr = ptr->next;
    }
    printf("\n");
}

void display_alternate(struct node *head) {
    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }
    struct node *ptr = head;
    printf("Alternate nodes: ");
    while (ptr != NULL) {
        printf("%d ", ptr->data);
        if (ptr->next != NULL) {
            ptr = ptr->next->next;
        } else {
            ptr = NULL;
        }
    }
    printf("\n");
}

void maxmin(struct node *head) {
    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }
    struct node *ptr = head->next;
    int max = head->data;
    int min = head->data;

    while (ptr != NULL) {
        if (ptr->data > max) max = ptr->data;
        if (ptr->data < min) min = ptr->data;
        ptr = ptr->next;
    }

    printf("Largest Element : %d\n", max);
    printf("Smallest Element: %d\n", min);
}

struct node* insert_node(struct node *head) {
    int value, choice, pos;

```

```

printf("Enter data of new node: ");
scanf("%d", &value);

struct node *newNode = (struct node*) malloc(sizeof(struct node));
if (newNode == NULL) {
    printf("Memory allocation failed.\n");
    return head;
}
newNode->data = value;
newNode->next = NULL;

if (head == NULL) {
    return newNode;
}

printf("1. At Beginning\n2. At Specific Position\n3. At End\n");
printf("Enter insertion type: ");
scanf("%d", &choice);

switch (choice) {
    case 1:
        newNode->next = head;
        head = newNode;
        break;

    case 2: {
        printf("Enter Position (>= 1): ");
        scanf("%d", &pos);

        if (pos <= 1) {
            newNode->next = head;
            head = newNode;
            break;
        }

        struct node *ptr = head;
        int count = 1;
        while (ptr->next != NULL && count < pos - 1) {
            ptr = ptr->next;
            count++;
        }

        newNode->next = ptr->next;
        ptr->next = newNode;
        break;
    }

    case 3: {
        struct node *ptr = head;
        while (ptr->next != NULL) {
            ptr = ptr->next;
        }
        ptr->next = newNode;
        break;
    }
}

```

```

    }

    default: free(newNode); return head;
}
display(head);
return head;
}

struct node* delete_node(struct node *head) {
    if (head == NULL) {
        printf("List is empty. Nothing to delete.\n");
        return NULL;
    }

    int choice, pos;
    printf("1. From Beginning\n2. From Specific Position\n3. From End\n");
    printf("Enter deletion type: ");
    scanf("%d", &choice);

    switch (choice) {
        case 1: {
            struct node *temp = head;
            head = head->next;
            free(temp);
            break;
        }

        case 2: {
            printf("Enter Position: ");
            scanf("%d", &pos);

            if (pos <= 1) {
                struct node *temp = head;
                head = head->next;
                free(temp);
                break;
            }

            struct node *ptr = head;
            int count = 1;
            while (ptr->next != NULL && count < pos - 1) {
                ptr = ptr->next;
                count++;
            }

            if (ptr->next == NULL) {
                printf("Position out of bounds.\n");
                return head;
            }

            struct node *temp = ptr->next;
            ptr->next = temp->next;
            free(temp);
            break;
        }
    }
}

```

```

    }

    case 3: {
        if (head->next == NULL) {
            free(head);
            head = NULL;
            break;
        }
        struct node *ptr = head;
        while (ptr->next->next != NULL) {
            ptr = ptr->next;
        }
        free(ptr->next);
        ptr->next = NULL;
        break;
    }

    default: return head;
}

display(head);
return head;
}

struct node* reverse_list(struct node *head) {
    struct node *prev = NULL, *current = head, *next = NULL;
    while (current != NULL) {
        next = current->next;
        current->next = prev;
        prev = current;
        current = next;
    }
    return prev;
}

void find_middle(struct node *head) {
    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }

    struct node *slow = head;
    struct node *fast = head;

    while (fast != NULL && fast->next != NULL) {
        slow = slow->next;
        fast = fast->next->next;
    }

    printf("Middle Element: %d\n", slow->data);
}

void free_list(struct node *head) {
    struct node *temp;

```

```
while (head != NULL) {  
    temp = head;  
    head = head->next;  
    free(temp);  
}  
}
```

## Output:

```
PS D:\DS\LAB3> gcc p1.c -o p1.exe
PS D:\DS\LAB3> .\p1.exe

1. Create Linked List
2. Display List
3. Display Alternate Nodes
4. Find Largest and Smallest Elements
5. Insert a Node
6. Delete a Node
7. Reverse Linked List
8. Find Middle Element
9. Exit
Enter your choice (1-9): 1
Enter Data: 10
Do you have another data? (y/n): y
Enter Data: 20
Do you have another data? (y/n): y
Enter Data: 30
Do you have another data? (y/n): y
Enter Data: 40
Do you have another data? (y/n): y
Enter Data: 50
Do you have another data? (y/n): n

Enter your choice (1-9): 2
List elements: 10 20 30 40 50

Enter your choice (1-9): 3
Alternate nodes: 10 30 50

Enter your choice (1-9): 4
Largest Element : 50
Smallest Element: 10

Enter your choice (1-9): 5
Enter data of new node: 25
1. At Beginning
2. At Specific Position
3. At End
Enter insertion type: 2
Enter Position (>= 1): 3
List elements: 10 20 25 30 40 50

Enter your choice (1-9): 6
1. From Beginning
2. From Specific Position
3. From End
Enter deletion type: 2
```

```
Enter Position: 3  
List elements: 10 20 30 40 50
```

```
Enter your choice (1-9): 7  
List elements: 50 40 30 20 10
```

```
Enter your choice (1-9): 8  
Middle Element: 30
```

```
Enter your choice (1-9): 9  
PS D:\DS\LAB3>
```

2. Write a program in C to Concatenate two Single Linked Lists.

**Source Code:**

```
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *next;
};

struct node* create_list();
void display(struct node *head);
struct node* concatenate(struct node *head1, struct node *head2);
void free_list(struct node *head);

int main() {
    struct node *head1 = NULL, *head2 = NULL, *head3 = NULL;

    printf("Enter elements for First Linked List:\n");
    head1 = create_list();

    printf("\nEnter elements for Second Linked List:\n");
    head2 = create_list();

    printf("\nFirst ");
    display(head1);

    printf("Second ");
    display(head2);

    head3 = concatenate(head1, head2);

    printf("\nConcatenated ");
    display(head3);

    free_list(head3);
    return 0;
}

struct node* create_list() {
    struct node *head = NULL, *ptr = NULL, *newNode = NULL;
    char ch;

    do {
        newNode = (struct node*) malloc(sizeof(struct node));
        if (newNode == NULL) {
            return head;
        }

        printf("Enter Data: ");
        scanf("%d", &newNode->data);
        newNode->next = NULL;
```

```

        if (head == NULL) {
            head = newNode;
            ptr = newNode;
        } else {
            ptr->next = newNode;
            ptr = ptr->next;
        }

        printf("Do you have another data? (y/n): ");
        scanf(" %c", &ch);
    } while (ch == 'y' || ch == 'Y');

    return head;
}

void display(struct node *head) {
    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }
    struct node *ptr = head;
    printf("List elements: ");
    while (ptr != NULL) {
        printf("%d ", ptr->data);
        ptr = ptr->next;
    }
    printf("\n");
}

struct node* concatenate(struct node *head1, struct node *head2) {
    if (head1 == NULL) return head2;
    if (head2 == NULL) return head1;

    struct node *ptr = head1;
    while (ptr->next != NULL) {
        ptr = ptr->next;
    }
    ptr->next = head2;
    return head1;
}

void free_list(struct node *head) {
    struct node *temp;
    while (head != NULL) {
        temp = head;
        head = head->next;
        free(temp);
    }
}

```

### Output:

```
PS D:\DS\LAB3> gcc p2.c -o p2.exe
PS D:\DS\LAB3> .\p2.exe
Enter elements for First Linked List:
Enter Data: 10
Do you have another data? (y/n): y
Enter Data: 20
Do you have another data? (y/n): n

Enter elements for Second Linked List:
Enter Data: 30
Do you have another data? (y/n): y
Enter Data: 40
Do you have another data? (y/n): n

First List elements: 10 20
Second List elements: 30 40

Concatenated List elements: 10 20 30 40

PS D:\DS\LAB3>
```